UAV Routing Problem for Maximum Information Collection with Energy Consumption and Time Windows

Alaittin Kırtışoğlu[1], Melis Boran[2], Mustafa Tural[2]

Abstract

Keywords: Iterated local search, Second order cone programming, UAV route planning

1 Introduction

In recent years, the application domains of unmanned aerial vehicles (UAVs) have expanded significantly beyond military to complex civilian and humanitarian missions. The capability of UAVs to navigate hazardous or difficult-to-access areas makes them essential for tasks such as post-disaster search and rescue, wildfire monitoring, and precision agriculture. The primary objective of these missions is often reconnaissance, in which the UAV must visit a set of potential targets to collect critical information.

Route planning is the basis for a UAV to carry out such missions, specifically referring to the decisions about which targets to visit and the order of their visits from the beginning to the destination within a mission area. Unlike traditional routing problems, where the value, i.e., the information associated with a target, is constant, targets in these missions often have time-varying values. For example, in these contexts, the information may represent the decaying survival probability of a person in a disaster zone, crop imaging that requires a specific midday solar window, or wildfire monitoring of an area where the information value is low before the fire reaches it and rises as the fire hits it. Therefore, the information collected at each target is dependent on the UAV's arrival time.

The time-dependent nature of information introduces a critical operational challenge: managing flight speed. Increasing speed enables the UAV to reach targets quickly and collect more information when information decays over time or when time windows are narrow, at the cost of substantial energy consumption. Conversely, reducing speed may allow the UAV to maximize information collection from distant targets by arriving at more productive times, though this does not necessarily result in energy savings. Thus, adjusting speed to maximize information collection creates a trade-off between time-dependent information and the UAV's limited energy reserves, i.e., battery capacity.

In this study, we address this trade-off by formulating a UAV routing problem that extends the classic Orienteering Problem with Time Windows (OPTW) framework to incorporate time-dependent prizes (specifically, information) and explicit energy constraints. The objective of our

[2]Department of Industrial Engineering, Middle East Technical University, Ankara, Turkey. E-mail: boranm@metu.edu.tr, tural@metu.edu.tr

[1]Department of Applied Mathematics, Illinois Institute of Technology, Chicago, IL 60616. E-mail: kirtisoglu@hawk.iit.edu

model is to maximize the total collected information by selecting an optimal subset of targets and determining a precise sequence of visits within a single mission operating under strict energy constraints. Contrary to the literature, the UAV's speed along each route segment is treated as an implicit continuous decision variable, directly coupling flight mechanics with mission outcomes.

The paper makes four main contributions: (i) we introduce and formally define a UAV routing problem that extends the OPTW by treating flight speed as an implicit decision variable and explicitly calculating energy consumption through an embedded non-linear propulsion model, (ii) we describe a mixed integer non-linear programming formulation (MINLP) for the problem that synchronizes arrival times with diverse time-dependent information profiles and show that it can be reformulated as a mixed-integer second-order cone programming (MISOCP) formulation, (iii) we develop a matheuristic approach that leverages the fixed-route speed optimization SOCP to efficiently solve large-scale problem instances while maintaining high solution quality, and (iv) we present extensive computational experiments on a dataset adapted from team orienteering problem with time windows (TOPTW), to quantify the impact of speed optimization on maximizing total collected profit.

The remainder of the paper is organized as follows. In the next section, we provide a review of the relevant literature. In Section 3, we formally define the UAV routing problem. We then describe a nonlinear mathematical formulation of the problem and present the reformulation as a second-order cone program. In Section , we introduce a matheuristic designed to solve large-scale problem instances. Section 5 presents results of computational experiments. Finally, Section 6 concludes the paper and discusses potential directions for future research.

2 Literature review

3 Problem Definition and Formulation

3.1 Problem description

Let $N = \{0, 1, \ldots, n\}$ denote a set of locations, where node 0 represents the depot and the remaining nodes $N \setminus \{0\}$ correspond to potential targets (points of interest). The Euclidean distance between any two locations $i, j \in N$ is denoted d_{ij}. The travel time between i and j depends on the UAV's flight speed v_{ij}, which is bounded by $v_{\min} \leq v_{ij} \leq v_{\max}$.

Each target $i \in N \setminus \{0\}$ is associated with a time window $W_i = [e_i, \ell_i]$, where e_i and ℓ_i denote the earliest and latest permissible arrival times. The information available at these boundaries is I_{e_i} and I_{ℓ_i}, respectively. The mission is conducted over a finite planning horizon $T_{\max}$. To quantify the information collected as a function of the arrival time a_i, we define a reward function

$$r_i(a_i) = \begin{cases} \gamma_i(a_i - e_i) + I_{e_i}, & \text{if } a_i \in W_i, \\ 0, & \text{otherwise,} \end{cases}$$

where $\gamma_i = (I_{\ell_i} - I_{e_i})/(\ell_i - e_i)$ is the rate of change. Depending on the sign of γ_i, the reward may increase or decrease within the window.

The problem is to select a subset of targets and determine a tour starting and ending at the depot that maximizes total collected information, subject to the following assumptions:

- Information rewards are additive across visited targets.
- Each target may be visited at most once.

- The UAV flies at a constant speed v_{ij} between consecutive targets. Loitering is modeled by allowing the flight path length L_{ij} to exceed the Euclidean distance d_{ij}; the UAV maintains speed v_{ij} along the extended path.

3.2 UAV platform and energy model

The Bayraktar TB2 is a medium-altitude long-endurance (MALE) fixed-wing unmanned aerial vehicle designed for intelligence, surveillance, and reconnaissance missions. Publicly available specifications, including mass, wing geometry, and performance limits, make the platform suitable as a representative case study for mission-level energy modeling and route planning optimization [2]. The manufacturer reports the following key specifications:

1. cruise speed $\approx 36\,\mathrm{m/s}$,
2. min–max speeds $\approx 20\,\mathrm{m/s}$ to $61\,\mathrm{m/s}$,
3. maximum endurance $\approx 27\,\mathrm{h}$.

Power consumption model. The propulsion power $P(v)$ required for a fixed-wing UAV to maintain steady level flight at speed v is modeled by Zeng et al. [6] as

$$P(v) = P_{\text{static}} + \frac{1}{2}\rho S C_{D,0}\, v^3 + \frac{2(mg)^2}{\rho S \pi e\, AR}\frac{1}{v}. \tag{1}$$

Defining the coefficients $c_0 = P_{\text{static}}$, $c_1 = \frac{1}{2}\rho S C_{D,0}$, and $c_2 = \frac{2(mg)^2}{\rho S \pi e\, AR}$, this simplifies to

$$P(v) = c_0 + c_1 v^3 + \frac{c_2}{v},$$

where c_0 represents the avionics (static) power, c_1 is the parasitic drag coefficient, and c_2 is the induced drag coefficient. For a mission segment of distance d, the energy consumption $E(v, d)$ is obtained by multiplying power by travel time $t = d/v$:

$$E(v, d) = c_0 \frac{d}{v} + c_1 d v^2 + c_2 \frac{d}{v^2}. \tag{2}$$

Model parameters and assumptions. The power consumption model is a simplified steady-flight representation intended for route planning and optimization, not detailed aerodynamic analysis. We adopt the parameters in Table 1 and validate the resulting energy model against official specifications reported by the manufacturer [2].

The model relies on the following assumptions:

- The UAV carries a fixed onboard imaging payload. The vehicle mass equals the manufacturer-reported maximum take-off weight, representing the fully equipped operational configuration.
- The UAV operates at a constant altitude of 16 000 ft, corresponding to the operational ceiling reported by the manufacturer [2]. Transient phases such as takeoff, climb, descent, and maneuvering are excluded.
- Propulsion power follows the form $P(v) = c_0 + c_1 v^3 + c_2/v$, capturing the trade-off between parasitic drag at high speeds and induced drag at low speeds. Aerodynamic properties and air density are assumed constant throughout the mission.

Table 1: Physical parameters for the Bayraktar TB2 power model.

Symbol	**Description**	**Value**	**Reference**
m_{MTOW}	Maximum take-off mass	700 kg	Baykar Tech [2]
b	Wingspan	12 m	Baykar Tech [2]
S	Wing reference area	12.135 m^2	Marotta [**?**]
AR	Aspect ratio	11.866	Calculated (b^2/S) [1]
$C_{D,0}$	Zero-lift drag coefficient	0.02	Marotta [**?**]
e	Oswald efficiency factor	0.8	Raymer [4]
ρ	Air density (16,000 ft)	0.746 $\mathrm{kg/m}^3$	ISA [3]
g	Gravitational acceleration	$9.806\,65\,\mathrm{m/s}^2$	[6]
c_0	Static (avionics) power	0 W	–

- Avionics power is set to $c_0 = 0$, so that the power model captures only the speed-dependent propulsion terms. This is conservative: including a positive c_0 would only increase the total energy consumption.

Characteristic speeds. Using the parameters in Table 1, the energy per unit distance $E(v, 1)$ from Equation (2) is convex in v, and two characteristic speeds emerge from the power–energy trade-off (see Figure 1).

Maximum range speed v_{mr}: minimizes energy per meter, maximizing flight distance for a given energy budget. Since $E(v, d) = P(v)\, d/v$,

$$v_{mr} = \arg\min_{v} \frac{P(v)}{v} = 44.31 \text{ m/s}.$$

Minimum power speed v_{mp}: minimizes energy per second, maximizing loiter endurance. Since $E(v, d) = P(v)\, t$,

$$v_{mp} = \arg\min_{v} P(v) = 33.67 \text{ m/s}.$$

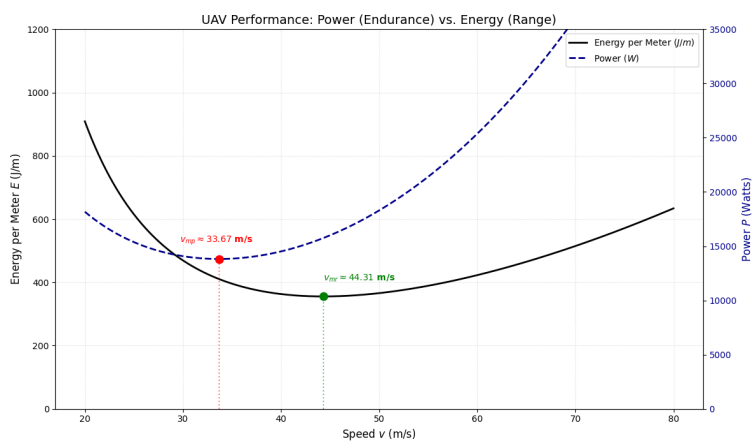

Figure 1: Maximum range speed versus minimum power speed.

3.3 MINLP formulation

To incorporate loitering into the formulation, we introduce the path length variable $L_{ij} \geq d_{ij}$ and define travel time as

$$t_{ij} = \frac{L_{ij}}{v_{ij}}.$$

Since L_{ij} may exceed d_{ij}, the travel time t_{ij} naturally accounts for any additional loitering time. Substituting $v_{ij} = L_{ij}/t_{ij}$ into the energy function (2) eliminates the speed variable and yields

$$\sum_{\substack{i,j \in N \\ i \neq j}} \left(c_0\, t_{ij} + c_1 \frac{L_{ij}^3}{t_{ij}^2} + c_2 \frac{t_{ij}^2}{L_{ij}} \right) \leq \mathcal{E}_{\max},$$

which, despite containing nonlinear terms, is amenable to a conic reformulation as shown in Section 3.4.

We now summarize the notation and present the MINLP formulation.

Sets and Indices

$N = \{0, 1, \ldots, n\}$: set of all locations (depot + targets)

$i, j \in N$, $i \neq j$ for any subscript ij

Parameters

d_{ij}: Euclidean distance between locations i and j

$T_{\max}$: mission time horizon (campaign time)

e_i: earliest arrival time at node i

ℓ_i: latest arrival time at node i

I_{e_i}: information available at time e_i for node i

γ_i: information slope for node i

$v_{\min}, v_{\max}$: minimum and maximum UAV speeds

c_0, c_1, c_2: energy consumption coefficients

$\mathcal{E}_{\max}$: energy budget

$\gamma_0 = I_{e_0} = e_0 = 0, \quad \ell_0 = T_{\max}$

Decision Variables

$$w_i = \begin{cases} 1 & \text{if node } i \text{ is visited,} \\ 0 & \text{otherwise} \end{cases}$$

$$x_{ij} = \begin{cases} 1 & \text{if the UAV travels from } i \text{ to } j, \\ 0 & \text{otherwise} \end{cases}$$

t_{ij}: travel time from i to j

a_i: arrival time at node i

L_{ij}: flight path length from i to j

MINLP Formulation

$$\max \quad \sum_{i \in N} \left[I_{e_i} w_i + \gamma_i (a_i - e_i w_i) \right] \tag{3}$$

subject to

$$w_0 = 1 \tag{4}$$

$$\sum_{\substack{j \in N \\ j \neq i}} x_{ji} = \sum_{\substack{j \in N \\ j \neq i}} x_{ij} = w_i, \qquad \forall\, i \in N \tag{5}$$

$$e_i w_i \leq a_i \leq \ell_i w_i, \qquad \forall\, i \in N \tag{6}$$

$$L_{ij} \geq d_{ij} x_{ij}, \qquad \forall\, i, j \in N,\ i \neq j \tag{7}$$

$$v_{\min} t_{ij} \leq L_{ij} \leq v_{\max} t_{ij}, \qquad \forall\, i, j \in N,\ i \neq j \tag{8}$$

$$a_j \geq a_i + t_{ij} - T_{\max}(1 - x_{ij}), \qquad \begin{smallmatrix} \forall\, i \in N \setminus \{0\}, \\ \forall\, j \in N,\ i \neq j \end{smallmatrix} \tag{9}$$

$$a_j \leq a_i + t_{ij} + T_{\max}(1 - x_{ij}), \qquad \begin{smallmatrix} \forall\, i \in N \setminus \{0\}, \\ \forall\, j \in N,\ i \neq j \end{smallmatrix} \tag{10}$$

$$a_j \geq t_{0j} - T_{\max}(1 - x_{0j}), \qquad \forall\, j \in N \setminus \{0\} \tag{11}$$

$$a_j \leq t_{0j} + T_{\max}(1 - x_{0j}), \qquad \forall\, j \in N \setminus \{0\} \tag{12}$$

$$t_{ij} \leq T_{\max} x_{ij}, \qquad \forall\, i, j \in N,\ i \neq j \tag{13}$$

$$\sum_{\substack{i,j \in N \\ i \neq j}} \left(c_0 t_{ij} + c_1 \frac{L_{ij}^3}{t_{ij}^2} + c_2 \frac{t_{ij}^2}{L_{ij}} \right) \leq \mathcal{E}_{\max} \tag{14}$$

$$w_i, x_{ij} \in \{0, 1\}, \quad a_i, t_{ij} \geq 0 \tag{15}$$

The objective (3) maximizes total collected information based on arrival times at visited nodes. Constraint (4) ensures the depot is always included. Flow conservation constraints (5) require that each visited node has exactly one incoming and one outgoing arc. Time window constraints (6) bound arrival times and force zero reward for unvisited nodes (since $a_i = 0$ when $w_i = 0$). Path length constraint (7) sets $L_{ij} \geq d_{ij}$ on active arcs, allowing loitering when $L_{ij} > d_{ij}$. Speed bounds (8) link path length and travel time. Time-precedence constraints (9)–(12) synchronize arrival times along the route, while constraint (13) forces $t_{ij} = 0$ on inactive arcs. The energy budget (14) limits total consumption, and (15) specifies variable domains.

Constraints (9)–(12) also eliminate subtours via a Miller–Tucker–Zemlin (MTZ) mechanism: since travel times are strictly positive on active arcs, arrival times increase monotonically along the route, preventing closed cycles. This monotonicity is relaxed only at the depot, which serves as the unique start and end point.

3.4 MISOCP reformulation

The energy constraint (14) contains nonlinear terms L_{ij}^3/t_{ij}^2 and t_{ij}^2/L_{ij} that prevent direct solution as a mixed-integer linear or conic program. We reformulate these terms using auxiliary variables and rotated second-order cones.

Since t_{ij} and L_{ij} are forced to zero on inactive arcs by constraints (8) and (13), the x_{ij} multipliers can be omitted from the energy constraint without affecting optimality. We introduce auxiliary variables:

- $z_{ij} \geq t_{ij}^2/L_{ij}$, representable directly as a rotated second-order cone;
- $s_{ij} \geq L_{ij}^2/t_{ij}$, an intermediate variable needed to decompose the cubic term;
- $y_{ij} \geq s_{ij}^2/L_{ij} = L_{ij}^3/t_{ij}^2$, completing the decomposition.

The MISOCP formulation replaces (14) with the following constraints:

$$\max \quad \sum_{i \in N} \left[I_{e_i} \, w_i + \gamma_i (a_i - e_i \, w_i) \right] \tag{16}$$

$$\text{subject to} \quad (4)\text{–}(13),\ (15),$$

$$\sum_{\substack{i,j \in N \\ i \neq j}} (c_0 \, t_{ij} + c_1 \, y_{ij} + c_2 \, z_{ij}) \leq \mathcal{E}_{\max} \tag{17}$$

$$\left\| \begin{pmatrix} t_{ij} \\ \frac{z_{ij} - L_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{z_{ij} + L_{ij}}{2}, \qquad \forall\, i, j \in N,\ i \neq j \tag{18}$$

$$\left\| \begin{pmatrix} L_{ij} \\ \frac{t_{ij} - s_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{t_{ij} + s_{ij}}{2}, \qquad \forall\, i, j \in N,\ i \neq j \tag{19}$$

$$\left\| \begin{pmatrix} s_{ij} \\ \frac{L_{ij} - y_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{L_{ij} + y_{ij}}{2}, \qquad \forall\, i, j \in N,\ i \neq j \tag{20}$$

$$y_{ij} \leq v_{\max}^3 \, t_{ij}, \qquad \forall\, i, j \in N,\ i \neq j \tag{21}$$

$$z_{ij} \leq t_{ij}/v_{\min}, \qquad \forall\, i, j \in N,\ i \neq j \tag{22}$$

$$s_{ij},\ y_{ij},\ z_{ij} \geq 0 \qquad \forall\, i, j \in N,\ i \neq j$$

Each cone constraint encodes a rotated second-order cone of the form $u^2 \leq ab$ (with $a, b \geq 0$):

- Constraint (18): $t_{ij}^2 \leq z_{ij} \cdot L_{ij}$, so $z_{ij} \geq t_{ij}^2/L_{ij}$.
- Constraint (19): $L_{ij}^2 \leq t_{ij} \cdot s_{ij}$, so $s_{ij} \geq L_{ij}^2/t_{ij}$.
- Constraint (20): $s_{ij}^2 \leq L_{ij} \cdot y_{ij}$, so $y_{ij} \geq s_{ij}^2/L_{ij} \geq L_{ij}^3/t_{ij}^2$.

The linearized energy (17) is exact at optimality since all auxiliary inequalities are tight when minimizing energy. Bounds (21) and (22) tighten the relaxation and ensure that auxiliary variables collapse to zero on inactive arcs, improving numerical stability and branch-and-bound performance.

4 Heuristic Approaches

4.1 Fixed-tour SOCP subproblem

Given a fixed route $R = (0, i_1, i_2, \ldots, i_k, 0)$ visiting k targets, the node selection and subtour elimination constraints are no longer needed. The remaining subproblem optimizes speeds (equivalently, travel times and path lengths) along the route to maximize information collection subject to the energy budget. Let $N_R = \{0, i_1, \ldots, i_k\}$ denote the nodes of the route and $A_R = \{(0, i_1), (i_1, i_2), \ldots, (i_{k-1}, i_k), (i_k, 0)\}$ its arcs.

Since all nodes in R are visited, the binary variables w_i and x_{ij} are fixed to one, and the arrival time constraints simplify to equalities. The resulting formulation is a continuous nonlinear program (NLP):

NLP Formulation

$$\max \quad \sum_{i \in N_R} \left(\gamma_i \, a_i - \gamma_i \, e_i + I_{e_i} \right) \tag{23}$$

subject to

$$\sum_{(i,j) \in A_R} \left(c_0 \, t_{ij} + c_1 \frac{L_{ij}^3}{t_{ij}^2} + c_2 \frac{t_{ij}^2}{L_{ij}} \right) \leq \mathcal{E}_{\max} \tag{24}$$

$$L_{ij} \geq d_{ij}, \qquad \forall \, (i,j) \in A_R \tag{25}$$

$$v_{\min} \, t_{ij} \leq L_{ij} \leq v_{\max} \, t_{ij}, \qquad \forall \, (i,j) \in A_R \tag{26}$$

$$e_i \leq a_i \leq \ell_i, \qquad \forall \, i \in N_R \tag{27}$$

$$a_j = a_i + t_{ij}, \qquad \forall \, (i,j) \in A_R \setminus \{(0, i_1)\} \tag{28}$$

$$a_{i_1} = t_{0,i_1} \tag{29}$$

$$t_{ij}, \, L_{ij} \geq 0 \qquad \forall \, (i,j) \in A_R \tag{30}$$

Compared to the MINLP formulation, the key simplifications are: (i) all nodes are visited, so $w_i = 1$ drops from the objective and the time windows become simple bounds (27); (ii) the tour sequence is fixed, so the big-M time-linking inequalities (9)–(12) reduce to equalities (28)–(29); and (iii) since $d_{ij} > 0$ for all arcs in A_R, constraint (26) ensures $t_{ij} > 0$, which in turn guarantees $a_j > a_i$ along the tour via (28).

Applying the same conic reformulation as in Section 3.4, we replace the nonlinear energy constraint with auxiliary variables and rotated second-order cones:

SOCP Formulation

$$\max \quad \sum_{i \in N_R} \left(\gamma_i \, a_i - \gamma_i \, e_i + I_{e_i} \right) \tag{31}$$

$$\text{subject to} \quad (25)\text{–}(30),$$

$$\sum_{(i,j) \in A_R} \left(c_0 \, t_{ij} + c_1 \, y_{ij} + c_2 \, z_{ij} \right) \leq \mathcal{E}_{\max} \tag{32}$$

$$\left\| \begin{pmatrix} t_{ij} \\ \frac{z_{ij} - L_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{z_{ij} + L_{ij}}{2}, \qquad \forall \, (i,j) \in A_R \tag{33}$$

$$\left\| \begin{pmatrix} L_{ij} \\ \frac{t_{ij} - s_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{t_{ij} + s_{ij}}{2}, \qquad \forall \, (i,j) \in A_R \tag{34}$$

$$\left\| \begin{pmatrix} s_{ij} \\ \frac{L_{ij} - y_{ij}}{2} \end{pmatrix} \right\|_2 \leq \frac{L_{ij} + y_{ij}}{2}, \qquad \forall \, (i,j) \in A_R \tag{35}$$

$$s_{ij}, \, y_{ij}, \, z_{ij} \geq 0, \qquad \forall \, (i,j) \in A_R$$

This SOCP subproblem is a continuous convex program solved efficiently by interior-point methods. It serves as the evaluation oracle for the local search: at each iteration, the metaheuristic proposes a new tour by modifying the node sequence, and the SOCP determines the optimal speeds and the tour's objective value.

4.2 Initial tour construction

The ILS framework (Section 4.6) requires an initial route R_0 as input. We generate four initial tours using different construction heuristics, run the ILS from each, and keep the best solution found across all starts. All heuristics verify feasibility at each construction step by solving the SOCP subproblem (Section 4.1) for the partial tour extended by the candidate node and the return to the depot. A candidate is accepted only if the resulting tour is SOCP-feasible.

R1. **Earliest-opening tour.** Starting from the depot, sort unvisited nodes by time-window opening e_i and select the first feasible one. Return to the depot after visiting 4 nodes.

R2. **Random-then-earliest tour.** Select the first node uniformly at random among feasible nodes in $N \setminus \{0\}$. Then sort remaining unvisited nodes by time-window opening e_i and select the first feasible one at each step. Return to the depot after visiting 4 nodes.

R3. **Random tour.** Starting from the depot, select 4 nodes uniformly at random from $N \setminus \{0\}$, accepting each only if the partial tour remains feasible. Return to the depot.

R4. **Information-efficiency tour.** Starting from the depot, iteratively extend the tour as long as possible. At each step, solve the SOCP for every candidate tour $R \cup \{j\}$ and select the node j^* that minimizes

$$j^* = \arg \min_{j \notin V(R)} \max \left\{ 1 - \frac{f(R \cup \{j\})}{I_{\max}}, \; \frac{E(R \cup \{j\})}{E_{\max}} \right\},$$

where $f(R \cup \{j\})$ and $E(R \cup \{j\})$ are the SOCP objective and total energy of the extended tour, $I_{\max} = \max_{i \in N \setminus \{0\}} \big(I_{e_i} + \gamma_i \cdot (\ell_i - e_i)\big)$ is the maximum collectible information at any node, and $E_{\max}$ is the energy budget. The criterion balances information gain and energy usage: a low value indicates a node that contributes high information relative to its energy cost. The tour extends until no feasible candidate remains.

Heuristics R1–R3 produce tours of fixed length 4, providing short starting points for the ILS to build upon. R4 adapts its length to the resource budgets and produces the longest feasible tour, giving the ILS a strong initial solution.

4.3 Neighborhood operators

To navigate the search space of the UAV routing problem, we define four neighborhood operators used within the iterated local search framework. Let $R = (r_0, r_1, \ldots, r_{|R|-1})$ denote the current route where $r_0 = 0$ is the depot, and let $\mathcal{C} = N \setminus (V(R) \cup \mathit{TabuList})$ denote the set of candidate nodes not currently in the route and not in the Tabu set *TabuList* (defined in Section 4.5).

- **Node Addition ($\mathcal{N}_{\mathbf{add}}$):** A node $j \in \mathcal{C}$ is selected and inserted at a random position $p \in \{1, \ldots, |R|\}$ in the current route. This allows the route to expand and explore new targets.
- **Node Replacement ($\mathcal{N}_{\mathbf{rep}}$):** A target node r_p ($p \neq 0$) is replaced by a node $j \in \mathcal{C}$. This performs a simultaneous removal and addition, maintaining a constant route length while improving the target selection.
- **Node Swap ($\mathcal{N}_{\mathbf{swap}}$):** Two positions $p, q \in \{1, \ldots, |R| - 1\}$ with $p \neq q$ are selected, and their corresponding nodes r_p and r_q are exchanged. This provides local reordering of mission nodes to improve time window compatibility.
- **2-opt Reversal ($\mathcal{N}_{\mathbf{2opt}}$):** Given a route $(r_0, \ldots, r_p, r_{p+1}, \ldots, r_q, \ldots, r_{|R|-1})$, the segment between positions p and q is reversed to produce $(r_0, \ldots, r_q, r_{q-1}, \ldots, r_p, \ldots, r_{|R|-1})$. This replaces the non-adjacent edges (r_{p-1}, r_p) and (r_q, r_{q+1}) with (r_{p-1}, r_q) and (r_p, r_{q+1}), serving as the primary mechanism for eliminating edge crossings and reducing travel distance.

Node removal is handled exclusively by the perturbation mechanism (Section 4.5), which removes multiple nodes simultaneously with Tabu-based exclusion to escape local optima more effectively than single-node removal within the local search.

The swap and 2-opt operators optimize the visiting sequence for distance and time efficiency, while addition and replacement modify which targets are visited. We use rejection sampling for the 2-opt operator to maintain a uniform distribution over all valid non-adjacent position pairs, preventing the search bias inherent in nested random selection.

4.4 Local move proposal

At each iteration of the local search, a single neighborhood operator is selected uniformly at random from the set of feasible operators and applied to the current tour. The set of available operators is determined by the tour length and the availability of candidate nodes, as described in Algorithm 1. The resulting tour is then evaluated by solving the SOCP subproblem (Section 4.1) to determine the optimal speeds and the corresponding objective value.

Algorithm 1: Local Move Proposal

Input: Current route R, Tabu set *TabuList*, depot node 0
Output: New route $R^\star$

$\mathcal{C} \leftarrow N \setminus (V(R) \cup \mathit{TabuList})$; `// Available candidate nodes`
$\ell \leftarrow |V(R)|$;
$\mathit{Moves} \leftarrow \emptyset$;

`// Filter operators based on tour size`
if $\ell = 1$ **then**
  $\mathit{Moves} \leftarrow \{\mathcal{N}_{\text{add}}\}$;
else if $\ell = 2$ **then**
  $\mathit{Moves} \leftarrow \{\mathcal{N}_{\text{add}}, \mathcal{N}_{\text{rep}}\}$;
else if $\ell = 3$ **then**
  $\mathit{Moves} \leftarrow \{\mathcal{N}_{\text{add}}, \mathcal{N}_{\text{rep}}, \mathcal{N}_{\text{swap}}\}$;
else
  $\mathit{Moves} \leftarrow \{\mathcal{N}_{\text{add}}, \mathcal{N}_{\text{rep}}, \mathcal{N}_{\text{swap}}, \mathcal{N}_{\text{2opt}}\}$;

`// Remove operators that require candidate nodes`
if $\mathcal{C} = \emptyset$ **then**
  $\mathit{Moves} \leftarrow \mathit{Moves} \setminus \{\mathcal{N}_{\text{add}}, \mathcal{N}_{\text{rep}}\}$;

$\mathit{Move} \leftarrow \text{UniformRandom}(\mathit{Moves})$;
$R^\star \leftarrow \text{Execute}(\mathit{Move}, R, \mathcal{C}, 0)$;

return $R^\star$; `// Triggers SOCP evaluation`

4.5 Perturbation

To prevent the search from stagnating in local optima, we implement a ruin-and-recreate perturbation mechanism. The *ruin* step removes a stochastic subset of k mission nodes from the current tour, while the *recreate* step leverages local search operators to rebuild and re-optimize the solution. This process is augmented by a Tabu tenure τ that prevents the immediate re-insertion of recently removed nodes, forcing exploration of different regions of the search space.

The Tabu set is maintained as a dictionary mapping each removed node to its expiry iteration. At iteration t, a node i is tabu if it was removed at some iteration t' satisfying $t' + \tau > t$. Expired entries are purged at the end of each ILS iteration (Algorithm 3, line 17). By excluding tabu nodes from the candidate set $\mathcal{C}$ in the local move (Algorithm 1), the mechanism prevents the algorithm from immediately re-inserting a node it just discarded, compelling it to explore different regions of the search space.

Algorithm 2: Perturbation (Ruin Step)

Input: Current route R, removal strength k, current iteration t, Tabu tenure τ
Output: Perturbed route R', new Tabu entries $\mathit{TabuList}_{\text{new}}$

$N_{\text{poss}} \leftarrow \{i \in V(R) \mid i \neq 0\}$;

`// Determine number of nodes to remove`

$m \leftarrow \min(k, |N_{\text{poss}}|)$;

$N_{\text{rem}} \leftarrow$ `RandomSample(`N_{poss}, m`)`;

`// Construct reduced route and update Tabu entries`

$R' \leftarrow [i \text{ for } i \in R \text{ if } i \notin N_{\text{rem}}]$;

$\mathit{TabuList}_{\text{new}} \leftarrow \{i \mapsto t + \tau \mid i \in N_{\text{rem}}\}$;

return R', $\mathit{TabuList}_{\text{new}}$

The combination of stochastic node removal with Tabu-based exclusion forms a classic "shake" mechanism in iterated local search. The removal strength k controls how aggressively the current solution is destroyed: small values make incremental changes, while larger values force more radical exploration. After perturbation, the local search phase (Algorithm 1) serves as the constructive phase, rebuilding the tour through a sequence of addition, replacement, and reordering moves.

4.6 Iterated local search

The overall solution method combines the SOCP evaluation oracle (Section 4.1), the local move operators (Section 4.3), and the perturbation mechanism (Section 4.5) into an iterated local search (ILS) framework. The algorithm alternates between a local search phase, which applies neighborhood operators to improve the current route, and a perturbation phase, which disrupts the route when the local search stagnates.

Stagnation is detected by counting consecutive non-improving iterations. When a local move is accepted (i.e., $f(R') \geq f(R)$), the counter resets to zero regardless of whether a new global best is found, since the search is still making progress. After θ consecutive rejected moves, the perturbation operator (Algorithm 2) removes k nodes and places them in *TabuList*, forcing the subsequent local search to reconstruct the route using different targets. Expired entries are purged from *TabuList* at the end of each iteration.

The acceptance criterion `Accept`(R', R) determines whether a proposed route replaces the current one. In our implementation, we use a hill-climbing criterion that accepts R' if and only if $f(R') \geq f(R)$, where f denotes the SOCP objective value. The best route encountered across all iterations is recorded separately.

Algorithm 3: Iterated Local Search with Tabu Perturbation

Input: Initial route R_0, max iterations $I_{\max}$, stagnation threshold θ, removal strength k, Tabu tenure τ

Output: Best route R_{best}

```
R ← R_0;
R_best ← R_0;
count ← 0 ;                                   // Iterations since last improvement
TabuList ← ∅;
for t ← 1 to I_max do
    if count ≥ θ then
        // Perturbation (shake)
        R′, TabuList_new ← Perturb(R, k, t, τ);
        TabuList ← TabuList ∪ TabuList_new;
        R ← R′ ;                              // Always accept perturbation
        count ← 0;
    else
        // Local search
        R′ ← LocalSearch(R, TabuList) ;                    // Algorithm 1
        if Accept(R′, R) then
            R ← R′;
            count ← 0 ;                       // Local improvement resets stagnation
            if f(R) > f(R_best) then
                R_best ← R;
        else
            count ← count + 1;
    TabuList ← {i : exp ∈ TabuList | exp > t} ;          // Purge expired entries
return R_best
```

The ILS framework has four hyperparameters: the maximum number of iterations $I_{\max}$, the stagnation threshold θ that triggers perturbation, the removal strength k that controls the intensity of the shake, and the Tabu tenure τ that determines how long removed nodes are excluded from re-insertion.

5 Computational Experiment

All experiments were conducted on an Apple M3 Pro processor using a native ARM64 Python 3.10 environment. The mathematical models were solved using Gurobi Optimizer (v11) with an academic license.

5.1 Solomon datasets

Solomon's VRPTW benchmark [5] comprises six problem sets: C1, C2, R1, R2, RC1, and RC2. In sets C1 and C2, customers are clustered in groups. In sets R1 and R2, they are uniformly distributed. Sets RC1 and RC2 are semi-clustered, containing a mix of randomly placed nodes and clusters. Problem sets R1, C1, and RC1 have a short scheduling horizon, while R2, C2, and

RC2 have a long one. The design of these test problems highlights the effect of time-window characteristics such as tightness and positioning.

Table 2: Solomon benchmark instances used in experiments.

Instance	Nodes	Spatial type	TW character	TW width (Solomon)	Campaign time
50_c101	50	Clustered	Wide	∼71	1236
50_r101	50	Random	Tight	10	230
r101	100	Random	Tight	10	230
rc101	100	Mixed	Medium	∼30	240
c101	100	Clustered	Wide	∼71	1236

R-type instances are structurally the hardest for our problem: tight windows (exactly 10 Solomon units) and random node positions mean that spatial and temporal structure do not reinforce each other. C-type instances have wide windows (∼71 units) but nodes are clustered, so geographic proximity partially compensates. RC-type instances sit between the two on both axes.

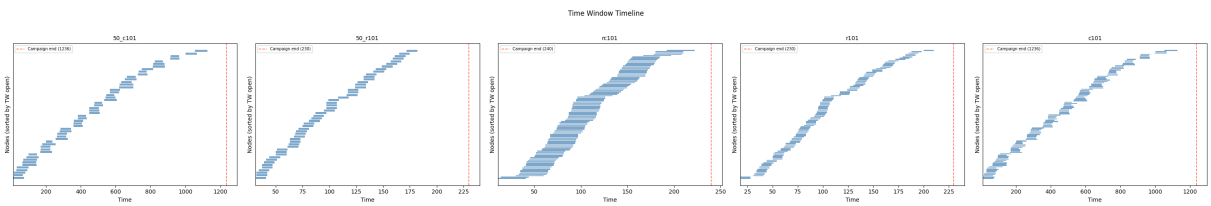

Figure 2: Information profiles by slope regime for Solomon dataset r101.

Visualize scheduling density (timeline), flexibility (width distribution), and constraint severity (tightness ratio). Dense overlapping TWs create scheduling conflicts; narrow widths limit speed optimization freedom. Tightness ratio is a single metric predicting solver difficulty.

5.2 Information decay and growth model

We defined I_{e_i} and I_{ℓ_i} in Section 3 as the information available at the opening and closing of the time window W_i, respectively. In our numerical experiments based on Solomon's benchmark datasets, we treat the maximum information value provided for each node as the initial information value I_{e_i}, available at the earliest arrival time e_i. Since the Solomon datasets do not contain any information content that we can relate to I_{ℓ_i} directly, we define a reward trajectory using the slope γ_i, which represents the rate of information change over time. To ensure physically and logically consistent rewards throughout the mission, we impose two boundary conditions at the latest arrival time ℓ_i to bound the permissible range of γ_i.

To ensure the reward remains a non-negative quantity, the information must not drop below zero within the window. To prevent unrealistic information inflation, we assume the reward at the end of the window cannot exceed twice its initial value. Combining these two bounds, we require

$$0 \leq r_i(\ell_i) \leq 2\, I_{e_i}. \tag{36}$$

Let $\Delta_i = \ell_i - e_i$ denote the duration of the time window W_i for node i. Based on the bounds (36), we uniformly sample the information slope γ_i from the symmetric interval

$$\gamma_i \sim \mathcal{U}\left(-\frac{I_{e_i}}{\Delta_i},\ \frac{I_{e_i}}{\Delta_i}\right), \tag{37}$$

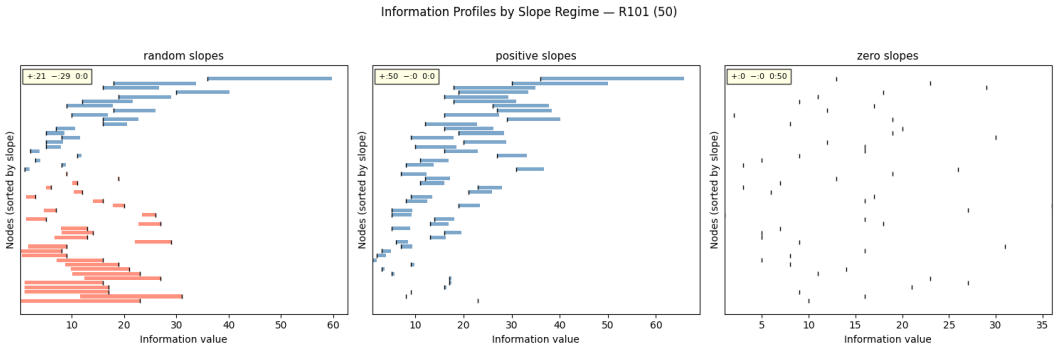

Figure 3: Information profiles by slope regime for Solomon dataset r101.

for every target node $i \in N$.

We consider three slope regimes to study the effect of information dynamics on solution structure:

1. **Mixed decay and growth.** Slopes sampled from the full interval (37), allowing both information decay ($\gamma_i < 0$) and growth ($\gamma_i > 0$).

2. **Growth only.** Slopes sampled from $\mathcal{U}(0,\ I_{e_i}/\Delta_i)$, so that information can only increase over time.

3. **Static information.** All slopes set to $\gamma_i = 0$, reducing the objective to the classical time-independent reward model.

Table 3 shows what the solver "sees" under each regime for the dataset r101. Random slopes create per-node early/late trade-offs; positive slopes uniformly reward late arrival; zero slopes make timing irrelevant. This explains the solver behavior differences in the exact solver experiments.

5.3 Mapping Solomon data to the real world

The Solomon VRPTW instances [5] provide geometric structure (node coordinates) and timing structure (time windows for each node), but they do not include UAV-specific operational parameters required by our SOCP formulation: the vehicle speed bounds $v_{\min}, v_{\max}$, the maximum propulsion energy for a tour $E_{\max}$. To obtain meaningful and reproducible experiments, we align each Solomon instance with a chosen UAV platform. We use the Bayraktar TB2 as a representative, well-documented fixed-wing UAV and calibrate physical parameters so that they preserve the benchmark structure and comparability.

Solomon's implicit speed. Solomon's VRPTW benchmark assumes a unit-speed vehicle: distances equal travel times ($d^s_{ij} = t^s_{ij}$), implying an implicit speed $v^s_{\text{opt}} = 1$ [5]. Our UAV platform operates at a maximum-range speed $v^d_{\text{opt}} = 44.31$ m/s. The calibration must therefore accomplish two things simultaneously: (i) map the Solomon campaign time to a physical sortie duration, and (ii) map the implicit Solomon speed to the drone's cruise speed.

Spatial and temporal scales. We introduce two transformation constants:

- *Temporal scale* β maps Solomon time units to seconds.

- *Spatial scale* α maps Solomon distance units to meters.

Under Solomon's convention, a vehicle traversing distance d^s takes time $t^s = d^s$. After scaling, the physical travel time is $\beta\, t^s$ and the physical distance is $\alpha\, d^s$. For the scaled speed to equal the drone's cruise speed, we require

$$v^d_{\text{opt}} = \frac{\alpha\, d^s}{\beta\, t^s} = \frac{\alpha}{\beta},$$

since $d^s = t^s$. This gives the coupling $\alpha = \beta \cdot v^d_{\text{opt}}$: the two scales are not independent but linked by the drone's speed.

Calibration procedure. We anchor the temporal scale on the Solomon campaign time and a prescribed sortie time T_{sortie} (set to 3 hours in our experiments). Let $c_{\max}$ denote the depot's time-window upper bound in the Solomon instance. The procedure is as follows:

1. Derive the temporal scale by mapping the Solomon campaign time to the prescribed sortie time:
$$\beta = \frac{T_{\text{sortie}}}{c_{\max}}.$$
2. Derive the spatial scale from the speed coupling: $\alpha = \beta \cdot v^d_{\text{opt}}$.
3. Scale all time windows: $W_i = [\beta\, e_i,\ \beta\, \ell_i]$ for every node $i \in N$. Scale all edge distances: $d_{ij} \leftarrow \alpha \cdot d_{ij}$.

This calibration simultaneously matches the campaign time to the UAV sortie duration and the implicit Solomon speed to the drone's cruise speed. The scaled time windows inherit their relative structure from the benchmark while carrying physical units (seconds). Because α and β depend only on $c_{\max}$ and the drone platform, the spatial and temporal scales are instance-independent for datasets sharing the same campaign time.

Energy budget. The calibration above determines the spatial and temporal scales but does not yet fix the energy budget $E_{\max}$. The natural baseline is the total energy available during the sortie at the cruise speed:

$$E_{\text{base}} = P(v^d_{\text{opt}}) \cdot T_{\text{sortie}}.$$

The TB2 platform has a reported maximum endurance of approximately 27 hours [2], so a 3-hour sortie uses only a fraction of the onboard energy capacity. While the sortie energy comfortably covers tours of moderate length at cruise speed, visiting all n targets would require energy well beyond E_{base}, so the constraint actively limits node selection and speed allocation. On the spatially compact Solomon instances, time windows are the tighter resource — this reflects the operational reality of long-endurance surveillance UAVs, where mission duration is governed by the planning horizon and time-sensitive targets rather than by energy depletion.

Energy scaling parameter. To study the effect of the energy constraint independently of the instance geometry, we introduce a scaling parameter $\eta \in (0, 1]$ that modulates the energy budget:

$$E_{\max}(\eta) = \eta \cdot P(v^d_{\text{opt}}) \cdot T_{\text{sortie}}.$$

At $\eta = 1$, the drone has the full sortie energy available. Decreasing η tightens the energy constraint while keeping the spatial and temporal scales (and hence all distances and time windows) fixed, isolating the effect of energy scarcity on tour structure, speed allocation, and node selection.

[TODO: Add scaled campaign times.]

Table 3: Calibration effects. All instances are mapped to a 3-hour sortie.

Instance	Solomon campaign	β (s/unit)	α (m/unit)	Scaled mean dist (m)	Scaled TW width (s)	$E_{\max}$ (J)
50_c101	1236	8.74	387.17	10,246	∼525	1.70×10^8
50_r101	230	46.96	2080.64	74,376	∼470	1.70×10^8
rc101	240	45.00	1993.95	89,200	∼1350	1.70×10^8
r101	230	46.96	2080.64	70,641	∼470	1.70×10^8
c101	1236	8.74	387.17	15,282	∼531	1.70×10^8

Scaled time-window widths. The raw Solomon time-window width is not a reliable comparator across instances; the *scaled* width is. For example, rc101 has a medium Solomon width (∼30) but a short campaign time (240), producing a large time scale (45 s/unit) as seen in Table 3 and therefore the widest scaled window (∼1350 s). This inversion illustrates that instance difficulty depends on the interplay between window width and campaign time after physical scaling.

5.4 Scalability of the exact solver

5.5 Effect of slope regime

5.6 Energy sensitivity

5.7 Value of loitering

5.8 Speed and arrival time distributions

5.9 Reproducibility

6 Conclusion

Future work discussions:

1. **Decomposed loitering and travel speeds.** In our current formulation, a single speed variable governs both loitering and travel on each arc. When loitering occurs, the entire arc is traversed at the minimum-power speed $v_{mp} = 33.67$ m/s, even though the energy-optimal speed for the non-loitering segment is $v_{opt} = 44.31$ m/s. This coupling masks the true energetic benefit of loitering. Introducing separate variables for loitering duration and travel speed on each arc would allow the solver to loiter at v_{mp} and cruise at v_{opt} independently. The resulting energy constraint $E_{\text{loiter}} + E_{\text{travel}} \leq E_{\max}$ remains SOCP-representable, preserving computational tractability.

2. **Stochastic information models.** Our linear reward trajectory $r_i(t) = I_{e_i} + \gamma_i(t - e_i)$ is deterministic. In practice, information availability at surveillance targets evolves stochastically, e.g., target presence follows a random process, sensor quality depends on weather. Replacing the deterministic slope with a stochastic model would yield robust routing policies. This introduces a stochastic ORTW.

3. **Heterogeneous multi-drone fleets.** Extending the single-drone model to multiple UAVs with different endurance, and speed ranges capabilities introduces a fleet routing and task allocation problem. Drones of varying size could be assigned complementary missions, e.g., a long-endurance platform for wide-area coverage and a small drone with rotary-wing for close inspection of high-value targets, resulting higher rewards with less energy consumption. This

connects to the heterogeneous vehicle routing literature while retaining the SOCP energy model per drone.

4. **Benchmark design for UAV routing.** Solomon's instances, designed for ground vehicles with unit speed, lack UAV-specific features such as altitude, no-fly zones, wind fields, and sensor footprints. A purpose-built benchmark incorporating these elements, along with realistic spatial scales and energy budgets, would advance reproducibility and enable meaningful comparison across the growing UAV routing literature.

5. **Tighter formulations and valid inequalities.** The MIP gap for larger instances (RC101-100 at xx %) suggests room for stronger formulations, e.g., subtour elimination cuts adapted for OPTW, or lifted inequalities exploiting the SOCP energy structure.

6. **Wind and altitude effects.** The power model assumes that UAV operates at constant altitude of 16 000 ft. Incorporating wind vectors and altitude-dependent air density or considering transient phases would make the energy model more realistic. The power curve shifts with wind, potentially breaking the SOCP structure, worth discussing as a modeling challenge.

References

[1] John D. Anderson, *Fundamentals of aerodynamics*, 6th ed., McGraw-Hill Education, New York, NY, 2016, Standard reference for Aspect Ratio (AR) and wing geometry relations.

[2] Baykar Technology, *Bayraktar tb2 tactical uav system: Technical specifications*, Tech. report, Baykar Makina, 2025, Accessed: 2025-12-19.

[3] International Civil Aviation Organization, *Manual of the icao standard atmosphere (extended to 80 kilometres)*, Tech. Report Doc 7488/3, ICAO, 1993, Primary source for Sea Level air density $\rho = 1.225$ kg/m^3.

[4] Daniel P. Raymer, *Aircraft design: A conceptual approach*, 5th ed., American Institute of Aeronautics and Astronautics, Inc., Reston, VA, 2012.

[5] Marius M Solomon, *Algorithms for the vehicle routing and scheduling problems with time window constraints*, Operations research **35** (1987), no. 2, 254–265.

[6] Yong Zeng and Rui Zhang, *Energy-efficient uav communication with trajectory optimization*, IEEE Transactions on Wireless Communications **16** (2017), no. 6, 3747–3760.